

E: Spacebar Tokenizer

Source file name: `spacebar.c`, `spacebar.cpp`, `spacebar.java`, or `spacebar.py`

Author: Juan Corena

Spacebar Tokenizer is a startup developing a state-of-the-art tokenizer. Its unique design helps the fastest programmers in the world avoid wasting time hitting the spacebar as they type.

To achieve this goal, the tokenizer uses a scoring system based on a large language model. This model assigns an integer score to each valid token in the language and a score of 0 to any token that is not recognized. When tokenizing a sentence, the tokenizer sums the scores of each token according to the model. An optimal tokenization is one that yields the highest total score out of all possible ways to tokenize the given sentence.

For example: *ilovespacebartokenizer* could be tokenized in several ways including, but not limited to: “*i love space bar tokenizer*”, “*i love spacebar tokenizer*”, and “*ilovespace bartokenizer*”. If the language model assigns a score of 5 to the token *spacebar*, 2 to the token *bar*, and 3 to the token *tokenizer*, then the second tokenization yields the highest score among the 3 example tokenizations, with a total score of 8. Note that there may be cases where multiple tokenizations result in the same maximum score.

The table below shows the scores for the sample tokenizations. The tokens contributing to the final score are in bold:

Tokenization	Score
i love space bar tokenizer	5
i love spacebar tokenizer	8
ilovespace bartokenizer	0

Your task is to create a program that outputs the score that a tokenizer should output, when it behaves optimally as described.

Input

The input consists of several test cases. Each test case consists of a language model providing the score for each token, and a series of sentences that need to be tokenized optimally according to the language model.

Each test case starts with a line containing two integers $1 \leq m \leq 1\,000$ and $1 \leq s \leq 100$ separated by a single space. Each of the next m lines contains, separated by a blank, a single token in undercase English alphabet and a positive integer number denoting the score that token adds to the overall score of the tokenization. The length of each token is at least 1 and at most 100 characters, the token can be assumed to be unique. Then s lines follow, each contains a sequence a_i for $1 \leq i \leq s$, of undercase English characters to be tokenized, where $1 \leq |a_i| \leq 1\,500$.

The input ends with two blank-separated 0s.

The input must be read from standard input.

Output

For each test case and a_i , output a single line with the optimal tokenization score of a_i according to the corresponding model.

The output must be written to standard output.

Sample Input	Sample Output
7 2 space 5 bar 7 spacebar 13 tokenizer 5 i 2 love 4 ipo 8 ilovespacebartokenizer theonlytokenizeripo 0 0	24 13